Claude Design brief: EPFL Machine Learning CS-433 project

PowerPoint content, visual direction and metric provenance

Create an editable **16:9 PowerPoint and a matching PDF** for cardiovascular-classification-numpy. Use **16 main slides and 3 technical appendix slides**. The deck must explain the project without a presenter, with readable captions and enough context to assess each result. The audience is Data Scientist hiring teams at product and technology companies.

Present this simply as an **EPFL Machine Learning CS-433 project**. Credit **Antonio Arbizzani Tonelli** and use a project-centered voice focused on methods, decisions and results. Use the portfolio's technical notes and recorded results; the academic submission report remains a separate artifact.

Narrative and design

Main message: The project combines a NumPy classification pipeline, controlled model comparisons and a documented implementation audit.

Lead with the outcome, then show the data problem, evaluation design, model choices, rejected alternatives and remaining limitations. Explain the reasoning behind specific decisions. Emphasize data correctness, useful metrics, reproducible analysis and clear reasoning. Keep the public deck company-neutral.

Use white evidence slides, navy text and a teal accent for the selected model. A restrained navy cover and closing can frame the deck. Use a readable sans-serif, 30-36 pt headlines and 20-24 pt body text. Prefer flat charts, tables and editable diagrams. Most slides should carry 40-70 visible words, with one main point. A caption must explain the result without relying on notes. Keep longer method details in the appendix and sources in notes.

Use `assets/chart_data/model_comparison.csv` for an editable chart, with a zero-based F1 axis from 0 to 0.50. The supplied PNG/SVG is a visual reference. Keep comparable development scores in one chart; show calibration and challenge scores separately. Do not add business impact, product deployment or an A/B test result that the project has not measured.

Ranking and score attribution

Use **“Ranked 7th on the AIcrowd leaderboard”**, followed by **“Leaderboard snapshot: 12 October 2026”**. The owner supplied this current position and the leaderboard link. Include a small note: **“Position and challenge score reported by the author.”** This is a dated position, not an asserted final placing. Do not invent a competition year, number of participants or percentile.

The original submission record describes AIcrowd F1 **0.452** and accuracy **0.878** as reported feedback. Keep that provenance distinct from numerical results reproduced in development evaluation.

Slide 1 - Cardiovascular disease classification in NumPy

On-slide copy:

> Antonio Arbizzani Tonelli
> EPFL Machine Learning CS-433 project
> Ranked 7th on the AIcrowd leaderboard · F1 0.452
> Leaderboard snapshot: 12 October 2026

Visual: A typographic cover with the project title leading, the course context below it and the rank beneath. Add the visible author-report note and a clickable leaderboard link.

Caption: A NumPy pipeline for preprocessing, model comparison and evaluation on BRFSS survey data.

Source: `evidence/METRICS.json`, `project/results/eda/final_phase14_submission.json`, owner confirmation and supplied leaderboard link.

Slide 2 - An end-to-end pipeline, from data inspection to model selection

On-slide copy:

> The workflow covers survey codebook inspection, preprocessing, NumPy models, validation, experiments and error analysis.
> The repository connects each model choice to its configuration, result record and limitations.

Visual: One compact workflow: inspect data to implement to evaluate to decide to audit. Pair each step with a repository location.

Caption: Each stage contributes evidence to the final model choice.

Source: `project/src/`, `project/configs/`, `project/docs/EXPERIMENTS.md`, `docs/CODE_REVIEW.md`.

Slide 3 - The data combined missing responses with a minority positive class

On-slide copy:

> 328,135 labeled respondents · 321 original predictors
> 8.83% positive labels · 44.79% missing predictor cells
> Target: reported myocardial infarction or coronary heart disease (`_MICHD`).

Visual: A labeled positive/negative proportion strip and a simple missingness statistic. Use a compact data description beside them; avoid a dashboard of unrelated indicators.

Caption: Missingness refers to the raw labeled predictor matrix. The task classifies a reported diagnosis in BRFSS 2015.

Source: `project/docs/DATA.md`, `project/results/eda/dataset_audit.json`, `evidence/METRICS.json`.

Slide 4 - Accuracy alone would hide failure on the positive class

On-slide copy:

Predicting every respondent as negative yields 91.17% accuracy and F1 0.
F1 guides model selection; precision, recall and average precision describe the trade-off.

Visual: Compare the all-negative baseline with the task's need to recover positives. Keep accuracy and F1 labeled as different metrics; do not put them on an unlabeled common scale.

Caption: A strong majority-class score is insufficient when the model identifies none of the positive respondents.

Source: `evidence/METRICS.json`, `project/src/evaluation.py`.

Slide 5 - Survey codes are transformed according to their meaning

On-slide copy:

295 retained source variables
`ALCDAY5=888`: no alcohol consumption to zero
`PHYSHLTH=88`: zero unhealthy days to zero
Feature-specific nonresponse codes to missing values

Visual: Three before/after examples with short explanations. Add a restrained count line: 1,095,421 nonresponse codes and 899,687 zero-quantity codes recoded on development rows.

Caption: Numeric codes can represent categories, missing answers or real quantities. The codebook determines the transformation.

Notes: Counts describe the 262,508-row development representation before binary recoding. They are data-transformation counts, not independently measured performance gains.

Source: `project/results/eda/phase33_preprocessing_counts.json`, `project/configs/experiments/phase14_codebook_corrections_only.json`.

Slide 6 - Fitting, threshold selection and evaluation use separate partitions

On-slide copy:

Three outer folds estimate development performance.
Two inner folds select each outer model's F1 threshold.
Preprocessing is fitted within the training partition of every fit.

Visual: One nested-fold diagram: training split, inner threshold selection and excluded outer evaluation. Label the boundaries directly.

Caption: The comparison uses 262,508 development rows. Threshold nesting controls threshold-selection optimism; repeated model choices still reuse the same development folds.

Source: `project/configs/experiments/final_phase14_validation.json`, `project/src/run_boosting_cv.py`, `evidence/METRICS.json`.

Slide 7 - Histogram gradient boosting implemented in NumPy

On-slide copy:

Quantile bins to Newton tree updates for logistic loss to summed tree predictions
Selected configuration: 200 trees, maximum depth 5.
Missing responses are routed inside each tree; binning is learned on training data.

Visual: A small annotated model diagram with one tree and an ensemble sum. Explain the operations in words; move equations and the full parameter table to the appendix.

Caption: Training uses NumPy and the Python standard library. The selected historical source is preserved for submission reproduction; the audited correction is shown later.

Source: `project/src/numpy_boosting_phase14.py`, `project/src/numpy_boosting.py`, `project/configs/final_model.json`.

Slide 8 - The selected booster improved development F1 by 0.0165

On-slide copy:

Historical logistic regression: 0.4256
Selected booster: 0.4422
Same development rows and outer folds; thresholds selected in inner folds.

Visual: Editable horizontal chart of the six models in `model_comparison.csv`, highlighting Phase 14. Keep the axis at 0-0.50 and use four decimal places in labels.

Caption: These are complete pipeline comparisons. Historical linear models use median-fill preprocessing, so the difference does not isolate model architecture alone.

Source: `assets/chart_data/model_comparison.csv`, `evidence/METRICS.json`.

Slide 9 - The tested neural networks did not beat the booster

On-slide copy:

MLP 64 to 32: F1 0.4245
MLP 128 to 64: F1 0.4193
Selected booster: F1 0.4422
Both networks underperformed the booster in all three outer folds.

Visual: Two small network sketches paired with a clear three-model comparison. Do not suggest that depth or width automatically implies a better model.

Caption: ReLU networks use binary cross-entropy and Adam, dedicated preprocessing and training-internal epoch selection.

Notes: The result concerns the tested architectures and configurations. It does not rule out better neural tuning on this dataset.

Source: `project/docs/MLP.md`, `project/results/eda/phase32_mlp_summary.md`, `project/src/numpy_mlp.py`.

Slide 10 - Additional changes did not improve the historical reference

On-slide copy:

Restricted imputation: 0.4418
Top 100 selected features: 0.4392
Positive class weight 1.25: 0.4408
Selected reference: 0.4422

Visual: A compact decision table: tested change, nested F1, decision. These variants use the historical Phase 14 control.

Caption: Negative experiments prevented added complexity from becoming the default without a measured benefit.

Notes: These results justify decisions among the tested settings, rather than a general claim that imputation, feature selection or weighting cannot work.

Source: `project/results/eda/phase24_original_restricted_imputation_summary.md`, `project/results/eda/phase29_feature_selection_summary.md`, `project/results/eda/phase35_positive_weights_summary.md`.

Slide 11 - Semantic cleaning had a small, uncertain measured gain

On-slide copy:

Raw predictors: F1 0.441457
Selected preprocessing: F1 0.442155
Difference: +0.000698
Paired 95% interval: [-0.001226, +0.002635]

Visual: One difference plot showing zero and the interval, with the two pipeline results beside it. Do not magnify the point estimate into a large improvement.

Caption: The interval includes zero. This comparison combines recoding and exclusions and is conditional on fixed fits and thresholds.

Notes: The raw pipeline has 321 predictors and the cleaned pipeline 295, with 128 candidate features per node in each. Semantic correctness supports cleaning; this benchmark establishes only a small uncertain predictive difference.

Source: `project/results/experiments/phase33_raw_preprocessing_comparison.json`, `project/results/eda/phase33_raw_preprocessing_summary.md`.

Slide 12 - The F1 result still involves missed cases and false positives

On-slide copy:

12,855 of 23,180 development positives identified
10,325 positives missed · 22,112 false positives
Precision 0.3676 · Recall 0.5546

Visual: A clearly labeled confusion matrix or two count bars. Keep positives recovered and false positives visually distinct.

Caption: These counts describe the selected nested development decisions. Choosing an operational threshold would require a defined use case and explicit error costs.

Notes: True negatives are 217,216. Do not translate these survey classification counts into patients helped, treatment decisions or business savings.

Source: `evidence/METRICS.json`, selected Phase 14 experiment record.

Slide 13 - A post-event question limits the prediction claim

On-slide copy:

`HAREHAB1` asks about rehabilitation after a heart attack.
All 620 development respondents with a recorded response are positive.
Removing it reduces historical F1 from 0.4422 to 0.4310.

Visual: A survey-question annotation and a paired with/without feature comparison. Make the feature timing visible.

Caption: The selected model classifies reported disease. Its result does not establish prediction before a cardiovascular event.

Notes: The final model retains this feature. A prospective task would need a new feature policy and validation design, including a timing audit of other variables.

Source: `project/docs/FINAL_REPORT.md`, `evidence/METRICS.json`, `project/results/experiments/20260921T154905753681Z_phase15-boosting-ablate-harehab1-depth5-features128-200trees.json`.

Slide 14 - An implementation audit led to a controlled comparison

On-slide copy:

The audit found omitted splits separating observed and missing responses.
Corrected Phase 34: F1 0.4408; AP 0.4288.
Historical Phase 14: F1 0.4422; AP 0.4286.

Visual: A small split sketch beside the two labeled implementations. Mark the historical submission as frozen and the corrected version as available.

Caption: The correction completes the candidate search. Its paired F1 difference interval includes zero; an algorithmic correction need not improve the chosen metric on this split.

Notes: Both versions were submitted. Reported challenge feedback contributed to retaining Phase 14; the numerical Phase 34 AIcrowd score was not supplied. Do not describe the selected historical implementation as the corrected algorithm.

Source: `project/results/eda/phase34_missing_only_split_summary.md`, `project/src/boosting_implementation.py`, `evidence/METRICS.json`.

Slide 15 - Development, calibration and challenge feedback measure different things

On-slide copy:

> Nested development: F1 0.4422 - excluded outer folds
> Full-train OOF calibration: F1 0.4428 - threshold selection
> Alcrowd feedback: F1 0.452 - reported challenge score

Visual: Three labeled rows with purpose, data scope and score. Do not plot them as successive improvements.

Caption: The final threshold is 0.2106169795070252. Its calibration labels also select that threshold, so calibration F1 is not an independent test.

Notes: Calibration covers all 328,135 labeled rows. The full-data model produces 109,379 predictions. Challenge feedback is used in final model selection; this is not a pristine external test reserved until all decisions were fixed.

Source: `project/results/eda/final_phase14_submission.json`, `project/configs/final_model.json`.

Slide 16 - A reproducible EPFL Machine Learning CS-433 project

On-slide copy:

> NumPy implementation, controlled experiments and a documented audit.
> Explore the code, configurations, results and audit on GitHub.
> Antonio Arbizzani Tonelli

Visual: A restrained closing with a clickable link to `https://github.com/antonio-arbizzani-tonelli/cardiovascular-classification-numpy`. Use one short project summary and an inspectable artifact map.

Caption: The project demonstrates data interpretation, implementation, validation and analytical judgment, with explicit limits on what its score supports.

Notes: Use the current test count and completed runtime checks in `evidence/VERIFICATION.json`. Distinguish local verification from Python 3.9 / NumPy 1.23.1 CI, and link to the repository's checks. Avoid claims of production deployment or absence of all bugs.

Source: `docs/CODE_REVIEW.md`, `evidence/VERIFICATION.json`, `project/README.md`.

Appendix A - The selected model is specified for reproduction

Use an editable parameter table: 200 trees; depth 5; learning rate 0.05; minimum 200 rows per leaf; 64 bins; 128 candidate features per node; leaf L2 1; positive weight 1; model seed 20260920; 295 source variables; final threshold 0.2106169795070252.

Show commands from `project/`: install `requirements-dev.txt`, run `python -m tools.verify_repository`, `python -m unittest discover -s tests -q`, then `python run.py` with official challenge data. Keep official data, checkpoints and row-level predictions outside Git. Explain that the source hash identifies the historical selected model.

Source: `project/configs/final_model.json`, `project/README.md`, `docs/CODE_REVIEW.md`.

Appendix B - Complete development metrics and provenance

Use the six-model table in `docs/RESULTS.md`: F1, precision, recall and average precision. Round values consistently; retain exact numerical data in `evidence/METRICS.json`. Label the nested protocol and historical linear preprocessing.

Add the current rank: **7th, owner-reported leaderboard snapshot on 12 October 2026**, with the supplied link. Keep the snapshot date and reported provenance readable. Do not present a final placing, cohort size or competition year without additional evidence.

Source: `docs/RESULTS.md`, `evidence/METRICS.json`, supplied leaderboard link.

Appendix C - Proposed validation for a new use case

Use a concise next-study sequence: define the decision and error costs; audit feature availability and timing; reserve fresh evaluation data; choose the operating threshold within training; evaluate performance and subgroup stability; measure drift and operational impact after deployment.

Label this as **future work**. These are proposed steps, not experiments or product results already demonstrated. Keep the current case study anchored in survey classification and the recorded challenge result.

Source: `docs/RESULTS.md`, `project/docs/DEVELOPMENT_PLAN.md`.

Delivery and publication

Return editable `RESULTS_PRESENTATION.pptx` and `RESULTS_PRESENTATION.pdf`, plus PNG previews of every slide. Use native charts and editable shapes. Check every number against `evidence/METRICS.json`, verify links and inspect all slides after export. Keep all visible text, captions, notes and filenames in English.

The exported PDF is the reading artifact; the PowerPoint is the editable source. Keep the deck's first two slides useful as a quick introduction, with the appendix supporting deeper review. Use the actual repository URL for the closing link:
`https://github.com/antonio-arbizzani-tonelli/cardiovascular-classification-numpy`.

Use this brief with the supplied figures, chart data, `docs/RESULTS.md` and technical notes. The companion PDF contains the design specification; the requested slide deck is a separate output. Keep the academic submission report outside the deck inputs.